



B2C API Documentation

Revision History

Date	Version	Author	Comments
17-03-2026	v1.0	Kennedy Gatimu	Initial document creation

Table of Contents

1.0 B2C Name Check API	4
Request URL	4
Request Field Description	4
Example Request Body	4
Responses	5
Success Response Body (200 OK)	5
Failure Response Body	5
2.0 B2C Transaction API	6
Request URL	6
Request Field Description	6
Example Request Body	7
Responses	8
Success Response Body (200 OK)	8
Failure Response Body	8
3.0 B2C Transaction Status Check API	9
Request URL	9
Request Field Description	9
Example Request Body	9
Responses	10
Success Response Body (200 OK)	10
Failure Response Body	10
4.0 Authentication	11
Signature Validation	11
Implementation	11
Algorithm	11
Canonical Text Construction	11
B2C Name Check (/api/v1/json/banktowallet/namecheck)	11
B2C Transaction (/api/v1/json/banktowallet)	11
B2C Transaction Status Check (/api/v1/json/banktowallet/status)	11
Java Signature Generation Sample	12
Private Key Loading Example	12

1.0 B2C Name Check API

This endpoint verifies a recipient wallet using MSISDN before initiating a transaction. It returns wallet details to prevent failed or misdirected transfers.

Request URL

POST https://10.11.6.110:8443/api/v1/json/banktowallet/namecheck

Request Field Description

Field	Type	Required	Description
receiver	Object	Mandatory	Contains recipient details
receiver.msisdn	String	Mandatory	Recipient mobile number. Pattern: ^\+?[0-9]+\$ Length: 5–20
receiver.telco	String	Mandatory	Telco name. Pattern: ^[a-zA-Z0-9-]+\$
channelInfo	Object	Mandatory	Contains channel metadata
channelInfo.rrn	String	Mandatory	Retrieval Reference Number. Must be 12 characters, alphanumeric
channelInfo.bankId	String	Mandatory	Bank identifier. Pattern: [a-zA-Z0-9-]+\$
channelInfo.channelId	String	Mandatory	Channel identifier. Pattern: [a-zA-Z0-9-]+\$
channelInfo.sourceSystemId	String	Mandatory	Source system identifier. Pattern: [a-zA-Z0-9-]+\$
timestamp	String	Mandatory	Request timestamp used in signature generation
signature	String	Mandatory	Digital signature. Max length: 1000 characters

Example Request Body

```
{
  "receiver": {
    "msisdn": "256783861192",
    "telco": "WENDI"
  },
  "channelInfo": {
    "rrn": "111996657000",
    "bankId": "54",
    "channelId": "OMNI",
    "sourceSystemId": "ONMI"
  },
  "timestamp": "31-Mar-2025 09:19:33",
  "signature":
"jZzbRrd8Ycwg8xClr0dddkZcHjXHnUtgf6FA9gMFtCMF720S8rZPiVgtc0nPBuNXa4WLYvK+elzapEDbxUvA+o
iyWtPZYM+fqI/UAXMjCtJifvr8z7ztj0x0Tq7La63Yvju06g6AW =="
```

Responses

A successful request returns a **200 OK** with the verified account holder's details.

Success Response Body (200 OK)

```
{
  "mobileNumberInquireResponse": {
    "statusCode": "00",
    "statusDescription": "Process service request successfully.",
    "customerName": "John Doe",
    "currency": "UGX",
    "telco": "WENDI",
    "mobileNumber": "256783861192"
  },
  "channelStatusInfo": {
    "rrn": "111996657000",
    "status": "SUCCESS",
    "code": "00",
    "description": "COMPLETED_SUCCESSFULLY."
  }
}
```

Failure Response Body

Namecheck is not blocking. A business logic or a downstream failure should not block the main transaction, and so, we always return a success response in channelStatusInfo block.

The **mobileNumberInquireResponse.statusDescription** tag will show the cause of failure for informational purposes.

```
{
  "mobileNumberInquireResponse": {
    "statusCode": "01",
    "statusDescription": "Process service request failed.",
    "customerName": "",
    "currency": "",
    "telco": "",
    "mobileNumber": ""
  },
  "channelStatusInfo": {
    "rrn": "111996657000",
    "status": "SUCCESS",
    "code": "00",
    "description": "COMPLETED_SUCCESSFULLY."
  }
}
```

2.0 B2C Transaction API

This endpoint processes a bank-to-wallet transaction request using a unified JSON payload. It validates input, verifies signature, and forwards the transaction for processing.

Request URL

POST <https://10.11.6.110:8443/api/v1/json/banktowallet>

Request Field Description

Field	Type	Required	Description
sender	Object	Mandatory	Contains sender details
sender.name	String	Mandatory	Sender full name
sender.account	String	Mandatory	Sender account number
sender.msisdn	String	Mandatory	Sender mobile number
receiver	Object	Mandatory	Contains receiver details
receiver.name	String	Mandatory	Receiver full name
receiver.account	String	Mandatory	Receiver account number
receiver.msisdn	String	Mandatory	Receiver mobile number
channelInfo	Object	Mandatory	Contains channel metadata
channelInfo.rrn	String	Mandatory	Unique 12-character transaction reference
channelInfo.bankId	String	Mandatory	Bank identifier
channelInfo.channelId	String	Mandatory	Channel identifier (e.g., OMNI)
channelInfo.sourceSystemId	String	Mandatory	Source system identifier
channelInfo.telco	String	Mandatory	Telco name
channelInfo.callbackUrl	String	Optional	URL for asynchronous callback
transactionInfo	Object	Mandatory	Contains transaction details
transactionInfo.amount	Decimal	Mandatory	Transaction amount (> 0)
transactionInfo.currency	String	Mandatory	3-letter currency code
transactionInfo.chargeFlag	String	Mandatory	Indicates if charges apply
transactionInfo.chargeAmount	Decimal	Mandatory	Transaction fee (>= 0)
transactionInfo.optIn	String	Mandatory	Customer opt-in flag
transactionInfo.transId	String	Mandatory	Unique transaction ID
transactionInfo.narration	String	Mandatory	Transaction description (max 100 chars)
timestamp	String	Mandatory	Request timestamp used in signature generation
signature	String	Mandatory	Digital signature (Base64 encoded, max 1000 chars)

Example Request Body

```
{
  "sender": {
    "name": "John Doe",
    "account": "12345678891",
    "msisdn": "256783861111"
  },
  "receiver": {
    "name": "Jane Doe",
    "account": "12345678000",
    "msisdn": "256783861192"
  },
  "channelInfo": {
    "rrn": "162996657000",
    "bankId": "54",
    "channelId": "OMNI",
    "sourceSystemId": "OMNI",
    "telco": "WENDI",
    "callbackUrl": "https://testurl.co.ke"
  },
  "transactionInfo": {
    "amount": 100.00,
    "currency": "UGX",
    "chargeFlag": "false",
    "chargeAmount": 0,
    "optIn": "false",
    "tranId": "TB25042800445720",
    "narration": "Deposit to wendi telco"
  },
  "timestamp": "31-Mar-2025 09:19:33",
  "signature":
    "jZzbRrd8Ycwg8xClr0dddkZchjXHnUtgf6FA9gMFtCMF720S8rZPiVgtc0nPBuNXa4WLYvK+elzapEDbxUvA+o
    iyWtPZYM+fqI/+T3XmFg2Kv5PdF7r0F0XvsaY9u6hfy2NbnQwaSMg=="
}
```

Responses

A successful request returns a **200 OK** with the verified account holder's details.

Success Response Body (200 OK)

```
{
  "bankToWalletTransactionResponse": {
    "telcoTransactionId": "100100",
    "telcoStatus": "OK",
    "telcoStatusDescription": "Transaction completed successfully"
  },
  "channelStatusInfo": {
    "rrn": "162996657000",
    "status": "SUCCESS",
    "code": "000",
    "description": "OPERATION_COMPLETED_SUCCESSFULLY"
  }
}
```

Failure Response Body

A failure indicates that the transaction was not completed successfully.

```
{
  "bankToWalletTransactionResponse": {
    "telcoTransactionId": null,
    "telcoStatus": "FAILURE",
    "telcoStatusDescription": "Transaction failed at telco"
  },
  "channelStatusInfo": {
    "rrn": "162996657000",
    "status": "FAILURE",
    "code": "01",
    "description": "Transaction failed"
  }
}
```

3.0 B2C Transaction Status Check API

This endpoint checks the status of a previously initiated bank-to-wallet transaction using the transaction reference number (RRN).

It returns the transaction outcome from the telco system

Request URL

POST https://10.11.6.110:8443/api/v1/json/banktowallet/status

Request Field Description

Field	Type	Required	Description
channelInfo	Object	Mandatory	Contains channel metadata
channelInfo.rrn	String	Mandatory	Unique 12-character transaction reference of the previous initiated bank-to-wallet transaction
channelInfo.bankId	String	Mandatory	Bank identifier
channelInfo.channelId	String	Mandatory	Channel identifier (e.g., OMNI)
channelInfo.sourceSystemId	String	Mandatory	Source system identifier
channelInfo.telco	String	Mandatory	Telco name
signature	String	Mandatory	Digital signature (Base64 encoded, max 1000 chars)

Example Request Body

```
{
  "channelInfo": {
    "rrn": "162996657000",
    "bankId": "54",
    "channelId": "OMNI",
    "sourceSystemId": "OMNI",
    "telco": "WENDI"
  },
  "signature":
  "jZzbRrd8Ycwg8xClr0dddkZchjXHnUtgf6FA9gMFtCMF720S8rZPiVgtc0nPBuNXa4wLYvK+elzapEDbxUvA+oiyWtPZYM+fqI/QHfjgMxvfkP+T3XmFg2Kv5PdF7r0F0XvsaY9u6hfy2NbnQwaSMg=="
}
```

Responses

A successful request returns a **200 OK** with the verified account holder's details.

Success Response Body (200 OK)

```
{
  "transactionStatusResponse": {
    "telcoTransactionId": "100100",
    "statusCode": "000",
    "status": "OK",
    "statusDescription": "Transaction completed successfully"
  },
  "channelStatusInfo": {
    "rrn": "162996657000",
    "status": "SUCCESS",
    "code": "000",
    "description": "OPERATION_COMPLETED_SUCCESSFULLY"
  }
}
```

Failure Response Body

```
{
  "transactionStatusResponse": {
    "telcoTransactionId": "",
    "statusCode": "400",
    "status": "ERROR",
    "statusDescription": "Generic Error"
  },
  "channelStatusInfo": {
    "rrn": "162996657000",
    "status": "SUCCESS",
    "code": "000",
    "description": "OPERATION_COMPLETED_SUCCESSFULLY"
  }
}
```

Important Note

- The **channelStatusInfo** block reflects the status of the API request processing
- The **transactionStatusResponse** block reflects the **actual** transaction status

Therefore:

- channelStatusInfo.status = SUCCESS means status check API worked correctly
- transactionStatusResponse.status = ERROR means transaction status retrieved from database or received from downstream telco.

4.0 Authentication

This Service API exclusively utilizes a digital signature for authenticating all requests to ensure message **integrity, authenticity, and non-repudiation**.

Signature Validation

Applicable Endpoints:

1. POST /api/v1/json/banktowelllet/namecheck
2. POST /api/v1/json/banktowelllet
3. POST /api/v1/json/banktowelllet/status

Implementation

The process involves:

1. Constructing a canonical plain-text string from request fields
2. Signing the string using a private key
3. Encoding the result using Base64
4. Sending the encoded signature in the request

Algorithm

1. SHA256withRSA
2. Base64 encoded

Canonical Text Construction

The canonical text is a simple concatenation of specific field values from the request body. These values **must be appended in the exact order specified below, with no separators or extra characters**.

B2C Name Check (/api/v1/json/banktowelllet/namecheck)

The canonical text is built by concatenating the following fields:

- **receiver.msisdn + timestamp + channelInfo.rrn**

B2C Transaction (/api/v1/json/banktowelllet)

The canonical text is built by concatenating the following fields:

- **transactionInfo.amount + sender.msisdn + sender.name + receiver.msisdn + channelInfo.telco + channelInfo.rrn**

B2C Transaction Status Check (/api/v1/json/banktowelllet/status)

The canonical text is built by concatenating the following fields:

- **channelInfo.rrn + channelInfo.channelId + channelInfo.bankId + channelInfo.telco**

Java Signature Generation Sample

```
public class WendiSignatureUtil {

    public static String generateSignature(TransactionRequest request, PrivateKey
privateKey) throws Exception {
        String clearText = buildCanonicalString(request);
        Signature signature = Signature.getInstance("SHA256withRSA");
        signature.initSign(privateKey);
        signature.update(clearText.getBytes(StandardCharsets.UTF_8));

        byte[] signedBytes = signature.sign();
        return Base64.getEncoder().encodeToString(signedBytes);
    }
    private static String buildCanonicalString(TransactionRequest req) {

        return req.getChannelInfo().getRrn()
            + req.getChannelInfo().getBankId()
            + req.getChannelInfo().getChannelId()
            + req.getSender().getAccountName()
            + req.getSender().getAccountNumber()
            + req.getReceiver().getAccountName()
            + req.getReceiver().getAccountNumber()
            + req.getTransactionInfo().getAmount()
            + req.getTransactionInfo().getFee()
            + req.getTransactionInfo().getTransactionCurrency();

    }
}
```

Private Key Loading Example

```
KeyStore keyStore = KeyStore.getInstance("PKCS12");
keyStore.load(new FileInputStream("keystore.p12"), "password".toCharArray());

PrivateKey privateKey = (PrivateKey) keyStore.getKey("alias",
"password".toCharArray());
```